

MODÜL 08

RNN, LSTM ve GRU

Recurrent Networks — Sequential Learning

Vanilla RNN · BPTT · LSTM 4 Kapı · GRU · Bidirectional · Seq2Seq

Seviye	Orta-İleri — Matematiksel Derinlik
Framework	NumPy (from scratch) · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 07 — Transfer Learning
Sonraki Modül	Modül 09 — Transformer ve Attention

Öğrenme Hedefleri

- ✓ Vanilla RNN forward pass ve BPTT gradyan hesabını açıklamak
- ✓ RNN'de vanishing gradient problemini matematiksel olarak kanıtlamak
- ✓ LSTM'in 4 kapısını (forget, input, cell, output) formülle açıklamak
- ✓ Cell state ile hidden state farkını açıklamak
- ✓ GRU ile LSTM arasındaki farkı parametre ve yapı açısından karşılaştırmak
- ✓ Bidirectional RNN'in avantajını açıklamak
- ✓ TF/Keras ve PyTorch'ta LSTM modeli kodlamak

Vanilla RNN

Formüller

Gizli durum: $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$

Çıkış: $y_t = W_{hy} \cdot h_t + b_y$

BPTT: $\partial L / \partial h_0 = \prod_t (\partial h_t / \partial h_{t-1})$ — her adımda tanh türevi çarpılır

Sorun

$\tanh$ türevi ≤ 1 , tipik $\sim 0.5 \rightarrow 50$ adım: $(0.5)^{50} \approx 10^{-15} \rightarrow$ ilk adımlar öğrenemiyor

LSTM Tam Analizi

Forget Gate

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ — Eski bilgiyi ne kadar unut? $f_t \approx 0$: unut, ≈ 1 : koru

Input Gate

$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ — Yeni bilgiyi ne kadar ekle?

Cell Update

$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ — Aday hücre değeri

Cell State

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ — UZUN VADELİ BELLEK — gradyan highway!

Output Gate

$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ — Çıkışı kontrol et

Hidden State

$h_t = o_t \odot \tanh(c_t)$ — KISA VADELİ BELLEK

GRU

Reset Gate

$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$ — Geçmişi ne kadar sıfırla?

Update Gate

$z_t = \sigma(Wz \cdot [h_{t-1}, x_t])$ — Eski/yeni oranı

Aday

$\tilde{h}_t = \tanh(Wh \cdot [r_t \odot h_{t-1}, x_t])$

Güncelleme

$h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ — Tek durum vektörü

LSTM vs GRU vs Vanilla RNN Karşılaştırması

Özellik	Vanilla RNN	LSTM	GRU
Kapı sayısı	0	4 (f,i,c,o)	2 (r,z)
Durum vektörü	h	h + c (iki)	h (bir)
Parametre	En az	En çok	LSTM'den az
Uzun bağımlılık	Zayıf	Güçlü	Güçlü
Hesap maliyeti	En düşük	En yüksek	LSTM'den az
Ne zaman	Kısa dizi, basit	Uzun dizi, karmaşık	Denge aranan durumlar

Framework Kullanımı

Model	TensorFlow / Keras	PyTorch
RNN	layers.SimpleRNN(64, return_sequences=True)	nn.RNN(10,64,batch_first=True)
LSTM	layers.LSTM(64, return_sequences=False)	nn.LSTM(10,64,batch_first=True)
GRU	layers.GRU(64)	nn.GRU(10,64,batch_first=True)
Bidirectional	layers.Bidirectional(LSTM(64))	nn.LSTM(10,64,bidirectional=True)
Çıkış (all)	return_sequences=True → (B,T,H)	output, (hn,cn) = lstm(x)
Çıkış (son)	return_sequences=False → (B,H)	hn[-1] son katman son adım

Yaygın Hatalar

Hata 1: return_sequences'ı yanlış seçmek	Dizi sınıflandırmada son adım yeterli (False). Seq2seq encoder'da tüm adımlar gerekli (True). Yanlış seçim boyut hatasına yol açar.
Hata 2: LSTM çıkışında h ve c'yi karıştırmak	PyTorch'ta lstm() → (output, (hn, cn)) döndürür. output tüm adımların h'si, hn son adımın h'si, cn son adımın c'sidir.
Hata 3: Bidirectional'da çıkış boyutunu yanlış hesaplamak	Bidirectional LSTM çıkışı 2×hidden_size boyutundadır. Linear katmanı 2×hidden_size giriş alacak şekilde ayarla.
Hata 4: Gradient clipping'i unutmak	RNN'ler exploding gradient'a özellikle açık. Her step sonrası: torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
Hata 5: Batch-first uyumsuzluğu	TF varsayılan: (batch, seq, feature). PyTorch varsayılan: (seq, batch, feature). batch_first=True ekle veya transpose kullan.

Kaynaklar

- [1] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- [2] Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder (GRU). EMNLP.
- [3] Pascanu, R. et al. (2013). On the Difficulty of Training Recurrent Neural Networks. ICML.
- [4] Graves, A. (2013). Generating Sequences with Recurrent Neural Networks. arXiv:1308.0850.
- [5] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning, MIT Press. Bölüm 10.
- [6] PyTorch RNN Documentation.
<https://pytorch.org/docs/stable/nn.html#recurrent-layers>

Sonraki Adım: Modül 09 — Transformer ve Attention